

Defense Script — Ecosystem Simulation

Yermek Aubayev (551098)

2026-09-14

Companion to `theory.md` (the concept-by-concept reference). This file is the rehearsal: what to say, what you'll likely be asked, and — since you may be asked to edit code live — exact, pre-verified recipes for the most likely “now add X” requests.

0. How to use this

Internalize the ideas, don't memorize the sentences. Reading from a script during an oral defense reads badly and won't survive a follow-up question phrased differently than you rehearsed. If you're asked to type code live, the recipes in §5 were checked line-by-line against the actual source on 2026-09-14 — re-open the real files and follow the pattern you see, rather than typing from memory under pressure.

Every “previously challenged” item below is from `docs/report/rebuttal.md` — four specific points the professor already raised once on this exact project. An examiner who caught something once very often re-probes it, to check it was genuinely understood and not just patched by someone else. Treat §3 as the highest-yield section to actually know cold.

1. Opening statement (30–45 seconds)

Say something close to this, in your own words:

“This project simulates predators, herbivores, and plants interacting directly on a shared grid — no central controller decides outcomes, each entity queries the world and acts on its own. It was built around two technical requirements at once: first, a genuinely discrete-event simulation engine — not a fixed-timestep loop that just calls itself one — and second, a runtime rule language that lets you author new ecosystem behavior, like a compound temperature rule, without touching or recompiling any Java source. Around those two pillars, the project demonstrates every core OOP principle with examples the system actually depends on to function, not ones added to satisfy a checklist.”

That last clause matters — it's a claim you need to be ready to back up anywhere the examiner points, not just in the sections you rehearsed.

2. Guided walkthrough — what to show, in order

1. **Entity hierarchy** (whiteboard or point at [class_diagram.puml](#)): `Entity` → `Organism` → `Animal` → {`Predator`, `Herbivore`}, `Plant` off `Organism` directly. State the one-sentence reason `Plant` isn't under `Animal` before being asked (theory.md A.3).
 2. **The FEL loop**: open `SimulationEngine.java` at `advanceTo()` (line 128). Read the loop out loud: pop earliest event, adopt *its* time, execute, repeat. Contrast explicitly with a `for` loop over all entities — that's the thing you're not doing.
 3. **One entity's lifecycle**: `EntityActivityEvent` → entity acts → reschedules its own successor. Point out this is why a dead entity's chain "just stops" — nothing reschedules for it.
 4. **The rule DSL, end to end**: open `config/rules.txt` or the in-app Rule Editor, show a compound rule (`CrowdedForest | Plant | stat.plantPopulation > 100 OR env.foodLevel < 20 | growthRate *= 0.5`), then open `RuleVocabulary.java` and show the `registerReadable` / `registerWritable` calls that back the exact tokens in that rule.
 5. **Multityping proof**: run the program, point at the console output before the GUI window appears — `Main.demonstrateMultityping()` prints five declared types for one object, ending in `entityRef == animalRef: true`.
 6. **Tests**: mention 107 tests, 13 classes, and name two or three specifically load-bearing ones (deterministic FEL tie-breaking, lazy invalidation, atomic rule-file rollback) rather than just the count.
-

3. High-risk questions — already raised once

3.1 “Aren't encapsulation and information hiding the same thing here?”

Don't repeat the old, now-fixed mistake of describing field visibility under “Encapsulation.” **Say:**

“They're different axes. Encapsulation is access control on one object's own state —

`World.getEntities()` returning a defensive copy so a caller can't mutate the internal list behind `World`'s back. Information hiding is whether a hierarchy's internal representation is free to change later — every field from `Entity` down to `Predator` is `private`, not `protected`, including in the abstract base classes, checked with the 3-step method.” Have the `World.java:85` and `Entity.java:24` line numbers ready (theory.md A.1–A.2).

3.2 “Show me your fields are actually private, not just your getters.”

This is the sharpest possible version of the question that was raised before — be ready for the professor to literally open a file and look. They are private (verified directly, current source):

`Entity.id/x/y/alive/ world/statistics/simulationEventListener/simulationTime/schedulingContext`, `Organism.energy/age/maxEnergy`, `Animal.speed/visionRange`, `Predator.attackPower`, `Herbivore.defensePower`. If asked *why this matters practically*, don't just recite the definition — say: “`private` and `final` together make `Entity.id` a **compiler-enforced** constant, not a documented convention someone could quietly break with a stray `this.id = ...` in a future subclass. `protected` would have let that happen and did, in one place, before the fix — `Predator.hunt()` used to read the inherited `visionRange` field directly instead of calling `getVisionRange()`.”

3.3 “Prove multityping from the client program, not the type declarations.”

Say: “`Main.demonstrateMultityping()`, called at the very start of `main()`. It builds one `Predator` and assigns that same reference to five variables of five different declared types — `Entity`, `Organism`, `Animal`, `Movable`, `Reproducible` — then calls a type-specific method through each one, and finally prints `entityRef == animalRef`, which is `true` because reference equality compares the underlying object, not the declared type of the variable holding it.” **Offer to run it live** — this is the strongest possible answer to this question, better than describing it.

3.4 “Your exception handling just defines exceptions, it doesn’t handle anything.”

Say: “Two real catch sites. First, `RuleEditorPanel.doReload()` — it catches `RuleParseException` from re-parsing a hand-edited `config/rules.txt` and explicitly keeps the previous in-memory rules rather than letting a corrupted file reach the running engine. That’s independently verified by a test, `RuleRepositoryTest.reloadingCorruptedFile_preservesLastValidRules`, not just asserted in prose. Second, `MainWindow`’s startup path used to catch the same exceptions but only write to `System.err` — invisible in a packaged Swing app, which isn’t really handling since the program’s visible behavior didn’t change at all. It now logs into the same on-screen event log the running simulation already uses for everything else.” If pressed on *what “handling” requires*: stop the crash, leave the program in one specific well-defined state, tell the user. A console-only catch satisfies only the first of those three.

4. Other likely questions

Grounded in the report’s own stated boundaries — an examiner who reads a “Known Limitations” section tends to probe exactly those lines, since they’re clearly the areas the author already knows are debatable.

- **“Why doesn’t the rule AST use polymorphism, like everything else here?”** → Deliberate non-example (theory.md A.5.1): the AST’s node set is fixed and closed by the grammar, so an exhaustive, compiler-checked `switch` over sealed `records` is the more type-safe tool than virtual dispatch, which would need an `accept() / visit()` pair on every node for the same guarantee pre-Java-21 tooling.
- **“Is the rule DSL Turing-complete?”** → No, deliberately not — no loops, no user-defined functions, no recursion, closed at the type level by the sealed AST, not just by convention. Say this before being asked; it’s in §4 of the report already.
- **“Why two interfaces, `Movable` and `Reproducible`, instead of one?”** → Interface Segregation: `Plant` is `Reproducible` but not `Movable`; forcing one interface would force `Plant` to implement a `move()` with no sensible body.
- **“Why `Math.random()` and not a seeded `Random`?”** → Stated limitation: exact run-to-run reproduction isn’t guaranteed; tests that depend on outcomes use batch-averaged or structurally deterministic assertions instead of exact single-run values, specifically because of this.
- **“Where exactly is this not perfectly modular?”** → Volunteer it:
`Statistics.adjustSpeciesCounter` and `EntityFactory.createOffspringNear`’s `instanceof` chain both independently enumerate the three concrete species. A fourth species needs both updated — a deliberate tradeoff (avoiding a registry abstraction that would be harder for a new reader to follow), not an oversight.

- “What’s the actual compile-time boundary of your ‘runtime customization’ claim?” → Composing new logic from what’s already registered needs zero Java changes. Exposing a genuinely new attribute/command needs one `registerX(...)` call and a recompile. Say both halves.
- “What happens if two events land on the exact same tick?” → `SimulationEvent.compareTo` breaks the tie on a monotonic `sequenceNumber` (an `AtomicLong`), so same-tick events always run in the order they were created — deterministic, because `PriorityQueue` alone doesn’t guarantee that.
- “What if an entity dies after an event referencing it is scheduled but before it runs?” → Lazy invalidation: no active search-and-cancel in the FEL (a `PriorityQueue` has no efficient arbitrary removal); instead every event re-checks `isAlive()` / `isPendingRemoval()` at the top of its own `execute()` and silently no-ops if stale. `pendingRemoval` is set synchronously the instant removal is *requested*, before the `DeathEvent` that actually flips `alive` runs.

5. “Now add this to the code” — live-coding recipes

Three tiers, cheapest first. If time is short, say so and do the cheapest one fully rather than the expensive one halfway.

5.1 Fastest (~2 min): a new rule-vocabulary attribute

Add to `RuleVocabulary.registerDefaults()` ([RuleVocabulary.java:194](#)) — e.g. a writable `visionRange` for `Animal`:

```
registerReadable("visionRange", Animal.class, ctx -> ((Animal)
    ctx.entity()).getVisionRange());
registerWritable("visionRange", Animal.class, (ctx, newValue) ->
    ((Animal) ctx.entity()).setVisionRange((int) clamp(newValue, 1, 30)));
```

Say while typing: “This is the Open/Closed extension point — one call here, nothing else changes. `Lexer`, `Parser`, `Rule`, `RuleEngine` are all untouched, and `RuleVocabularyValidationTest` / `ParserTest` already prove that pattern by adding coverage against the vocabulary without ever touching those files.” Then show it works: a rule like `Cautious | Animal | age > 50 | visionRange += 5` should now parse and validate.

5.2 Medium (~3–4 min): a new discrete-event type

Extend `SimulationEvent`, implement the two abstract methods. Minimal skeleton (matches the shape of the existing six event classes):

```

package com.ecosystem.simulation.events;

import com.ecosystem.simulation.entities.Entity;

public class MigrationEvent extends SimulationEvent {

    private final Entity entity;

    public MigrationEvent(int scheduledTime, Entity entity) {
        super(scheduledTime, entity);
        this.entity = entity;
    }

    @Override
    public void execute(SchedulingContext ctx) {
        if (!entity.isAlive() || entity.isPendingRemoval()) {
            return; // lazy invalidation – same pattern as every other event
        }
        // ... the actual state mutation this event owns goes here ...
    }

    @Override
    public String getDescription() {
        return entity + " migrated";
    }
}

```

Say while typing: “Same lazy-invalidation guard every other event opens with. `EventQueue` and `SimulationEngine.advanceTo()` need zero changes — they only ever interact with events through the abstract `SimulationEvent` base type, which is inclusion polymorphism doing real work, not decoration.” Schedule one with `ctx.schedule(new MigrationEvent(ctx.getClock() + 10, this))` from inside whichever event/entity should trigger it.

5.3 Full (~8–10 min): a new species

Use **Scavenger** if the professor doesn’t name a species — `Reproducible.java`’s own Javadoc already names `Scavenger` as the example hypothetical new species, so it’s consistent with what’s already in the codebase’s comments. Keep behavior minimal on purpose (move + consume energy + age + reproduce, no hunting/grazing interaction) so it’s actually typeable under time pressure.

Step 1 — the class itself (`entities/Scavenger.java`), same shape as `Herbivore.java` minus the grazing logic:

```

package com.ecosystem.simulation.entities;

import com.ecosystem.simulation.events.ReproductionEvent;

public class Scavenger extends Animal implements Reproducible {

    private int reproductionCooldown;
    private int maxAge = 60;
    private int energyConsumptionPerTick = 1;
    private int reproductionCooldownPeriod = 15;
    private int populationCap = 15;

    public Scavenger(int x, int y, int energy, double speed) {
        super(x, y, energy, speed);
        this.reproductionCooldown = 0;
        setVisionRange(15);
    }

    @Override
    public void update() {
        if (!isAlive()) {
            return;
        }

        move();
        consumeEnergy(energyConsumptionPerTick);
        if (!isAlive() || isPendingRemoval()) {
            return;
        }

        increaseAge();
        if (getAge() >= maxAge) {
            scheduleRemoval("old age");
            return;
        }

        if (reproductionCooldown > 0) {
            reproductionCooldown--;
        }
        if (getEnergy() > getMaxEnergy() * 0.6 && reproductionCooldown == 0) {
            reproduce();
        }
    }

    public void setMaxAge(int maxAge) { this.maxAge = maxAge; }
    public void setEnergyConsumptionPerTick(int amount) { this.energyConsumptionPerTick = amount; }
    public void setReproductionCooldownPeriod(int period) { this.reproductionCooldownPeriod = period; }
    public void setPopulationCap(int cap) { this.populationCap = cap; }
    public int getPopulationCap() { return this.populationCap; }

    @Override
    public void reproduce() {
        if (getWorld() == null || getSchedulingContext() == null) {
            return;
        }
        if (getWorld().countAliveByType("Scavenger") >= populationCap) {
            reproductionCooldown = 10;
            return;
        }
        int reproductionCost = getEnergy() / 2;
        consumeEnergy(reproductionCost);
        if (!isAlive() || isPendingRemoval()) {

```

```

        return;
    }
    getSchedulingContext().schedule(new
    ReproductionEvent(getSchedulingContext().getClock(), this));
    this.reproductionCooldown = reproductionCooldownPeriod;
}
}

```

Narrate while typing: “ extends `Animal` for free energy/age/speed/vision management, implements `Reproducible` — not `Movable` directly, because `Animal` already provides that. `move()` is inherited concretely from `Animal`, so I don’t override it unless I want different movement.”

Step 2 — wire it into `EntityFactory` ([EntityFactory.java:36](#)):

```

public Scavenger createScavenger(int x, int y) {
    Scavenger s = new Scavenger(x, y, 60, 1.8); // literals for the demo;
    wire(s); // see Step 5 for the config-driven
    version
    return s;
}

```

...and add one branch to `createOffspringNear`’s `instanceof` chain ([EntityFactory.java:78](#)):

```

if (parent instanceof Scavenger) {
    return createScavenger(newX, newY);
}

```

Step 3 — register the target type in `RuleVocabulary.TARGET_TYPES` ([RuleVocabulary.java:56](#)):

```

private static final Map<String, Class<? extends Entity>> TARGET_TYPES = Map.of(
    "Entity", Entity.class,
    "Organism", Organism.class,
    "Animal", Animal.class,
    "Predator", Predator.class,
    "Herbivore", Herbivore.class,
    "Plant", Plant.class,
    "Scavenger", Scavenger.class // new
);

```

Needs one new import at the top of this file too — `import com.ecosystem.simulation.entities.Scavenger;` (the file imports each entity class individually, no wildcard). `EntityFactory.java` needs no new import since it already lives in the `entities` package with `Scavenger`.

Say this explicitly — it’s a genuine, precise nuance, not a mistake to hide: “The report’s extensibility section says a new species needs ‘one new `EntityFactory.createX(...)` method plus matching config keys, no engine change.’ That’s true for the FEL and the event dispatch mechanism. It’s not quite the whole story for *this* step — the rule grammar’s `targetType` token is validated against a fixed compile-time map, `TARGET_TYPES`, so if I want `rules.txt` to be able to say `Scavenger | ...`, that map needs one more entry too. Small, but worth being precise about rather than reciting the report’s simplified version.”

Step 4 — spawn some at startup, in `SimulationEngine.initialize()` ([SimulationEngine.java:81](#)):

```
for (int i = 0; i < config.initialScavengers(); i++) {
    spawnInitial(entityFactory.createScavenger(randomPos(world.getWidth()),
        randomPos(world.getHeight())));
}
```

...plus one `SimulationConfig` getter (same pattern as `initialPredators()` / `initialHerbivores()` / `initialPlants()`):

```
public int initialScavengers() { return getInt("population.scavenger", 5); }
```

Say this too — the sharper version of the extensibility claim: “This is the one place I’d push back slightly on the report’s own phrasing. ‘The engine needs no changes’ is true for how an *existing* entity is driven once it exists — the FEL and `advanceTo()` genuinely don’t care what type popped off the queue. It’s not true for how the species is *bootstrapped* — without this loop, a `Scavenger` would need some other entity to reproduce one, and there isn’t one yet. That’s a real, small gap in the simplified claim, and I’d rather name it than have it found.”

Step 5 — optional, if time allows (full parity with existing species): config-driven construction instead of the literals in Step 2 (add `scavengerEnergy()` / `scavengerSpeed()` getters to `SimulationConfig`, same `getInt` / `getDouble` pattern), and a case in `Statistics.adjustSpeciesCounter` ([Statistics.java:130](#)) if birth/death counters should track scavengers specifically. **Note:** population-based rule references (`stat.scavengerPopulation`, if you added one) would already work without touching `Statistics` at all — `stat.*` readables call `world.countAliveByType("Scavenger")` directly on `World`, which counts live entities generically by class simple name.

6. If you don’t know the answer

- Restate the question back in your own words — it buys thinking time and confirms you understood it.
- Reason from the design principles you do know (Liskov, Interface Segregation, Open/Closed) rather than guessing at a specific line of code you can’t recall exactly.
- If the honest answer is “that’s a known limitation,” say so and point at §4 of the report — that section exists specifically so you’re not caught flat-footed by a boundary you already stated.
- Never invent a claim to fill a gap. A precise “I’d need to check that” is a better answer than a confident wrong one, and it’s recoverable; a fabricated claim that gets checked live is not.

7. Don’ts — overclaims to avoid

- Don’t claim “no other project on GitHub does this” or anything like it, even if it comes up. The honest, defensible version is narrower: among the handful of comparable student/hobby projects actually read, none combined a genuine FEL with a runtime rule DSL — that is not the same as a claim about GitHub as a whole, and overstating it is exactly the kind of thing that invites a sharp follow-up you can’t back up.
- Don’t claim the rule DSL is more general than it is (§4: not Turing-complete, by design).

- Don't claim perfect modularity — volunteer the `Statistics / EntityFactory` duplication yourself (§4 above) rather than waiting to be caught.
- Don't claim exact run-to-run reproducibility — `Math.random()` is used directly, stated as a limitation.
- Don't restate the pre-rebuttal versions of §3.1–§3.4 from memory; the code and the report were both changed specifically because those versions were wrong or incomplete.

8. Final checklist before walking in

☐

Can recite, without notes: encapsulation vs. information hiding, in two different sentences.

☐

Can name which fields were `protected` before the fix, and why `protected` was the actual problem (reachable from subclass *and* package, not just “not private enough” vaguely).

☐

Can run `Main.main()` and point at the multityping console output live.

☐

Can find and read aloud both `MainWindow` catch blocks and the `RuleEditorPanel.doReload()` catch block without searching for them.

☐

Can type the Tier-1 rule-vocabulary recipe (§5.1) from memory in under 2 minutes.

☐

Have `theory.md` Part B and Part C open or printed — DES and DSL theory are the two “central technical differentiator” claims from the report’s own introduction, and are the most likely deep-dive target.